■ Chat with Git

Project Explanation & File Reference

A conversational AI application that ingests any public GitHub repository and lets you ask natural-language questions about the codebase using Google Gemini and LlamaIndex RAG.

Generated on	October 13, 2026 15:40
Stack	Python · Streamlit · LlamaIndex · Google Gemini · GitIngest
Location	d:\projects\Chat with GIT

1. Project Overview

Chat with Git is a Retrieval-Augmented Generation (RAG) web application built with Streamlit. It allows a developer to load any public GitHub repository by URL and then ask plain-English questions about that codebase. The application:

- Fetches and parses the entire repository source tree using GitIngest.
- Splits the Markdown-formatted content into nodes using LlamaIndex's MarkdownNodeParser.
- Embeds each node into a vector store using Google's Gemini Embedding model (gemini-embedding-001).
- Answers user questions by retrieving the most relevant nodes and passing them to the Gemini 3.5 Flash Lite LLM, which synthesises a grounded answer.
- Streams the response token-by-token into the Streamlit chat interface.

Architecture Diagram (Conceptual)

2. File-by-File Explanation

app.py — Main Application (Google Gemini)

This is the primary entry-point for the production-ready version of the application. It runs with `streamlit run app.py` and uses Google Gemini for both text generation and embeddings.

Imports & Initialisation (lines 1–29)

Standard library modules (os, gc, sys, tempfile, uuid) are imported first. The script forcibly reconfigures stdout to UTF-8 to avoid encoding errors on Windows. `load_dotenv()` reads the GOOGLE_API_KEY from the `.env` file silently so the key is never displayed in the UI.

Constants & Custom Exception (lines 31–37)

MAX_REPO_SIZE (250 MB) and SUPPORTED_REPO_TYPES define constraints. GitHubRAGError is a custom exception that provides clean, user-friendly error messages throughout the application instead of raw Python tracebacks.

Utility Functions (lines 39–78)

validate_github_url(url)

Checks that a URL starts with https://github.com/ or http://github.com/.

get_repo_name(url)

Strips trailing slashes, splits on '/' and returns the last segment as the repo name.

reset_chat()

Clears the Streamlit session-state message list and runs garbage collection.

process_with_gitingest(github_url)

Calls gitingest.ingest() which downloads the repository, walks the file tree, and returns three values: summary (one-line description), tree (file structure string), and content (full Markdown dump of all source files concatenated).

get_gemini_models(model_name, api_key)

Decorated with @st.cache_resource so Gemini LLM and Embedding objects are created only once per Streamlit session, avoiding repeated API handshakes. Returns a (GoogleGenAI, GoogleGenAIEmbedding) tuple.

create_query_engine(content_path, repo_name, tree) (lines 80–118)

This is the core RAG pipeline function:

- SimpleDirectoryReader loads all Markdown files from the temp directory.
- MarkdownNodeParser splits the documents into semantically meaningful chunks respecting heading boundaries.
- VectorStoreIndex.from_documents() embeds all nodes using the Gemini embedding model and stores them in an in-memory FAISS-like index.
- A custom QA prompt template is created with {tree}, {context_str}, and {query_str} placeholders.

- The tree variable is pre-filled with partial_format(tree=tree) so the repository structure is always injected into every prompt.
- The index is converted to a streaming query engine and returned.

Session State Initialisation (lines 120–128)

Streamlit re-runs the entire script on every user interaction. Session state acts as persistent memory across reruns. The app stores: id (unique per browser tab), file_cache (dict mapping repo keys to query engines), messages (chat history), current_repo (active repo name), and current_summary.

Page Config & API Setup (lines 130–140)

st.set_page_config sets the browser tab title to 'Chat with Git' and the emoji favicon. The Google API key and model name are read from the environment — they are never rendered into the UI, keeping secrets out of the browser.

Sidebar UI (lines 142–210)

The sidebar contains only the repository URL input and a 'Load Repository' button. When clicked: the API key is validated, URL is validated, the Gemini models are initialised (or retrieved from cache), and process_with_gitingest is called inside a temporary directory. The resulting query engine is cached in session_state.file_cache with a key combining the session UUID and repo name, so different tabs or repos don't clash.

Main Chat UI (lines 221–308)

The main area shows the page title, a 'Clear Chat' button, previous messages, and three suggested-question buttons when a repo is freshly loaded. The chat_input widget captures the user's query. On submission the query engine is retrieved from cache and queried. Streaming chunks are appended to a placeholder element in real time. The complete response is then saved to session_state.messages for display on the next rerun.

app_local.py — Local / Offline Version (Ollama + HuggingFace)

This is the original offline version of the application, designed to run entirely on local hardware without any paid API calls. It was part of the upstream open-source project and is kept for reference.

LLM: Ollama (llama3.2)

Instead of calling Google's API, it connects to a locally running Ollama server on port 11434. The model used is llama3.2 (Meta's Llama 3.2 series). Ollama must be installed and running separately before starting this version.

Embeddings: HuggingFaceEmbedding

Uses BAAI/bge-large-en-v1.5, a high-quality open-source sentence embedding model downloaded from HuggingFace Hub. The first run will download ~1.3 GB of model weights.

Key Differences vs app.py

- No API keys required — fully private, runs offline.
- Requires Ollama server to be running locally.
- Uses HuggingFace sentence-transformers for embeddings instead of Gemini.
- Less structured error handling (no GitHubRAGError class).
- Also writes a local content.md debug file to disk.
- Run with: streamlit run app_local.py

.env — Environment Variables

This file stores secret configuration values and is never committed to version control (it is listed in .gitignore). The application reads it automatically at startup via python-dotenv's load_dotenv().

```
GOOGLE_API_KEY="&lt;your-google-api-key-here&gt;"
```

The GOOGLE_API_KEY is issued from Google AI Studio (aistudio.google.com). It authenticates calls to the Gemini LLM and Gemini Embedding API. Without this key the application will show an error and refuse to process repositories.

■■ Never share or commit this file. The .gitignore file already excludes it from Git.

.env.example — Template for Environment Setup

A safe, committed placeholder that shows other developers what environment variables are needed without exposing real values. New contributors should copy this file to .env and fill in their own credentials:

```
GOOGLE_API_KEY="your-google-api-key"
```

requirements.txt — Python Dependencies

Declares all Python packages needed to run the application. Install with:

```
pip install -r requirements.txt
```

Package	Purpose

`gitingest`	Converts a GitHub repository URL into a structured Markdown dump of the source code.
`streamlit`	The web framework. Provides the chat UI, sidebar, widgets, and session state without any HTML/JS.
`python-dotenv`	Loads .env files into os.environ at startup so secrets are available as environment variables.
`pandas`	Data manipulation library (imported for potential future data display features).
`llama-index`	Core RAG framework providing document loading, indexing, retrieval, and query orchestration.
`llama-index-llms-google-genai`	LlamaIndex integration for Google's unified GenAI SDK — provides the GoogleGenAI LLM class.
`llama-index-embeddings-google-genai`	LlamaIndex integration for Gemini embeddings — provides the GoogleGenAIEmbedding class.
`google-genai`	Google's official Python SDK for GenAI (used under the hood by the llama-index integrations).
`llama-index-llms-ollama`	LlamaIndex integration for local Ollama LLMs (used by app_local.py).
`llama-index-embeddings-huggingface`	HuggingFace sentence-transformer embeddings (used by app_local.py).
`huggingface-hub`	Downloads pre-trained models from HuggingFace (used by app_local.py's bge-large-en-v1.5).

README.md — Project Documentation

The README is the first file a developer reads when they open the project. It contains:

- A brief one-line description of the project.
- Installation instructions: pip install -r requirements.txt.
- Environment setup: how to create .env with GOOGLE_API_KEY.
- Run commands: streamlit run app.py (Google Gemini) or streamlit run app_local.py (Ollama).

.gitignore — Git Exclusion Rules

Prevents sensitive and auto-generated files from being committed to Git:

```
.env
__pycache__/
*.pyc
.ipynb_checkpoints/
content.md
```

.env contains the secret API key. __pycache__ and *.pyc are Python bytecode caches. content.md is a debug file written by app_local.py. .ipynb_checkpoints are Jupyter artefacts.

run_app.bat & run_app.ps1 — One-Click Launchers

Convenience scripts so the user does not have to type environment variable flags each time. Both scripts set PYTHONIOENCODING=utf-8 and PYTHONUTF8=1 before launching Streamlit, which prevents Unicode encoding errors on Windows terminals.

run_app.bat (Windows Command Prompt / Double-click)

```
@echo off
set PYTHONIOENCODING=utf-8
set PYTHONUTF8=1
echo Starting Chat with Git (Google Gemini RAG)...
streamlit run app.py
pause
```

run_app.ps1 (PowerShell)

```
$env:PYTHONIOENCODING = "utf-8"
$env:PYTHONUTF8 = "1"
Write-Host "Starting Chat with Git (Google Gemini RAG)..." -ForegroundColor Cyan
streamlit run app.py
```

3. End-to-End Data Flow

The following describes the complete journey of a user interaction, from entering a GitHub URL to receiving an AI-generated answer.

Step 1 — Enter Repository URL

User pastes a GitHub URL (e.g., https://github.com/tiangolo/fastapi) into the sidebar input and clicks 'Load Repository'.

Step 2 — GitIngest Ingestion

process_with_gitingest() calls gitingest.ingest(url). GitIngest clones or fetches the repository, walks all source files, and returns: (a) a summary string, (b) an ASCII file-tree string, and (c) one large Markdown document with all file contents concatenated under their filenames as headings.

Step 3 — Write to Temp File

The Markdown content is written to a .md file inside a Python tempfile.TemporaryDirectory, which is automatically deleted after indexing. This prevents disk clutter.

Step 4 — Document Loading

SimpleDirectoryReader reads the .md file and creates LlamaIndex Document objects.

Step 5 — Node Parsing

MarkdownNodeParser splits documents at heading boundaries, creating one Node per logical code section. Each node keeps its parent heading as metadata for context.

Step 6 — Embedding & Indexing

VectorStoreIndex.from_documents() calls GoogleGenAIEmbedding.get_text_embedding() for every node via the Gemini Embedding API. Each 3072-dimensional embedding vector is stored in an in-memory vector index.

Step 7 — Query Engine Creation

The index is wrapped as a streaming query engine with a custom prompt template that always injects the repository file-tree so the LLM has structural context.

Step 8 — Cache & Display

The query engine is saved in st.session_state.file_cache so subsequent questions skip re-ingestion and re-indexing. A success banner is shown in the sidebar.

Step 9 — User Asks a Question

The user types a question in the chat input. The text is appended to the message history and displayed in the chat bubble.

Step 10 — Retrieval

The query engine embeds the question, computes cosine similarity against all node embeddings, and retrieves the top-K most relevant code sections.

Step 11 — Generation

The retrieved nodes are formatted into the QA prompt template and sent to GoogleGenAI (gemini-3.5-flash-lite). The LLM generates a grounded answer.

Step 12 — Streaming Response

As tokens arrive, they are accumulated in a string and the Streamlit placeholder is updated in real time showing the growing response with a cursor '■'. Once complete, the full response is saved to session state.

4. Technology Stack

Technology	Version / Model	Role
Python	3.9+	Runtime language
Streamlit	1.63	Web UI framework — chat interface, sidebar, session state
GitIngest	0.3.x	GitHub repository → Markdown converter
LlamaIndex Core	0.14.x	RAG orchestration: document loading, indexing, querying
Google GenAI SDK	2.30.x	Underlying SDK for Google Gemini API calls
GoogleGenAI LLM	gemini-3.5-flash-lite	Answer generation — fast, cost-effective Gemini model
GoogleGenAI Embed	gemini-embedding-001	3072-dim embeddings for semantic search
python-dotenv	1.x	Load secrets from .env file
Ollama (local)	llama3.2	Alternative offline LLM for app_local.py
HuggingFace	BAAI/bge-large-en-v1.5	Alternative offline embeddings for app_local.py

5. How to Run the Application

Prerequisites

- Python 3.9 or later installed.
- A Google AI Studio account with a valid API key.
- Internet access (for GitHub fetching and Gemini API calls).

Step 1 — Install Dependencies

```
pip install -r requirements.txt
```

Step 2 — Configure API Key

Create or edit the .env file in the project root:

```
GOOGLE_API_KEY="your-actual-key-here"
```

Step 3 — Start the App

Option A — double-click run_app.bat

Option B — PowerShell:

```
.\run_app.ps1
```

Option C — Terminal:

```
streamlit run app.py
```

Step 4 — Use the App

- Open http://localhost:8501 in your browser.
- Paste a public GitHub repository URL in the sidebar.
- Click 'Load Repository' and wait for ingestion (30s–2min depending on repo size).
- Type your question or click a suggested prompt.
- Receive a streaming, grounded answer about the codebase.